

Recomendación de Clean Architecture para SISTEMA POS

Proyecto: Sistema POS e inventario para Provisiones Gurima

Autor: Manus AI

Fecha: 24 de agosto de 2026

1. Recomendación ejecutiva

Recomiendo construir el sistema como un **monolito modular con Clean Architecture, organización por módulos de negocio y una aplicación de escritorio Wails delgada**. La API Go centralizada debe ser el único punto de entrada para autenticación, autorización, reglas de negocio, transacciones y acceso a MySQL. Los clientes Wails deben limitarse a presentar la interfaz, gestionar el estado visual y actuar como clientes HTTPS de la API.

Esta decisión es más adecuada que una arquitectura de microservicios para la primera versión. El alcance actual contempla una sucursal, una caja física y un número reducido de usuarios, mientras que las operaciones críticas —venta, cobro, caja, inventario, crédito, facturación y auditoría— necesitan consistencia transaccional y una operación sencilla. Un monolito modular conserva límites claros entre capacidades, pero evita introducir despliegues múltiples, mensajería distribuida, consistencia eventual y observabilidad innecesaria para el tamaño inicial del negocio.

La arquitectura debe ser **modular desde el primer día, pero no distribuida**. Si en el futuro crecen las sucursales, aparecen integraciones fiscales o se incorpora analítica avanzada, los módulos podrán evolucionar o extraerse porque sus reglas y puertos ya estarán aislados.

Decisión central: Clean Architecture + modular monolith + vertical slices por módulo de negocio; no microservicios para la primera versión.

La propuesta respeta el baseline aprobado: Svelte y TypeScript en la interfaz, Wails y Go como cliente local, API Go centralizada, HTTPS/TLS y JSON en la comunicación, `database/sql` y `sqlc` para persistencia, MySQL centralizado y `golang-migrate` para migraciones. ¹ También responde al alcance funcional de catálogo, POS, caja, inventario, compras, crédito, devoluciones, facturación, reportes, auditoría, migración y respaldos. ²

2. Por qué esta arquitectura encaja con el proyecto

El sistema no es únicamente una pantalla de ventas conectada a tablas. Sus operaciones tienen reglas y efectos coordinados: una venta puede disminuir inventario, registrar uno o varios pagos, afectar caja, crear una cuenta por cobrar, emitir un comprobante y generar

una entrada de auditoría. Una recepción parcial de compra debe incrementar existencias solo por lo recibido y conservar una incidencia. Una devolución debe decidir si el producto vuelve al inventario vendible o se registra como merma. Por tanto, la lógica no debe quedar dispersa entre componentes Svelte, handlers HTTP y consultas SQL.

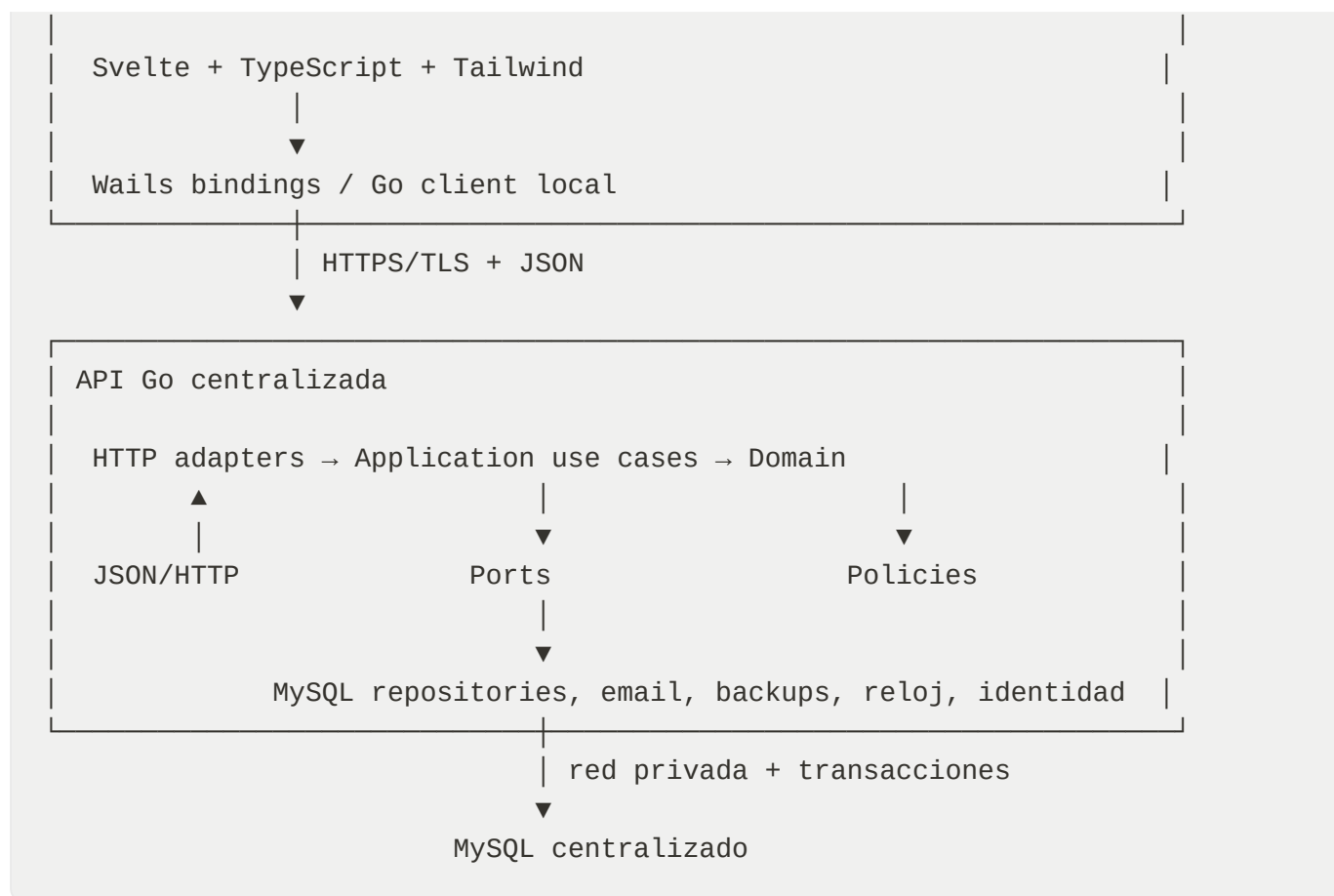
La documentación también exige que los descuadres de caja, ajustes de inventario, anulaciones, devoluciones, mermas y autorizaciones queden trazados sin alterar silenciosamente la operación original. ² Esto favorece un núcleo de dominio con operaciones explícitas, movimientos inmutables y casos de uso que controlen las transacciones completas.

Necesidad del proyecto	Decisión arquitectónica	Beneficio
Una venta afecta varias áreas	Caso de uso transaccional en la API	La operación se confirma completa o se revierte completa
La seguridad no puede depender del cliente	Autorización en API y políticas de aplicación	Un usuario no puede evadir restricciones modificando la aplicación local
Se necesita trazabilidad	Libro de movimientos y auditoría append-only	Se conserva la historia de ventas, caja, inventario y crédito
El negocio es pequeño en la primera versión	Monolito modular	Menor complejidad operativa y menor coste de mantenimiento
Se prevén impresión e integraciones futuras	Puertos y adaptadores	Las integraciones se incorporan sin contaminar el dominio
MySQL y sqlc son decisiones aprobadas	Persistencia detrás de interfaces	El dominio no depende de SQL ni de tipos generados
Reportes y operación tienen necesidades distintas	CQRS ligero dentro del mismo backend	Consultas eficientes sin convertir el sistema en microservicios

3. Forma general de la arquitectura

El flujo físico recomendado es el siguiente:





La regla de dependencia debe apuntar siempre hacia el centro: **el dominio no conoce HTTP, Svelte, Wails, MySQL ni sqlc** ; **la aplicación conoce puertos, pero no implementaciones**; los adaptadores traducen protocolos y datos; la infraestructura conecta las implementaciones concretas.

3.1 Capas propuestas

Capa	Responsabilidad	Puede conocer	No debe conocer
domain	Entidades, agregados, objetos de valor, invariantes y errores de negocio	Tipos y reglas propias del negocio	HTTP, JSON, MySQL, sqlc , Wails, Svelte
application	Casos de uso, comandos, consultas, autorización de operación y coordinación transaccional	Dominio y puertos/interfaces	Detalles de MySQL, handlers y componentes visuales
adapters	HTTP, serialización JSON, presenters, mapeadores y	Application y contratos externos	Reglas de negocio duplicadas

	adaptadores de entrada/salida		
infrastructure	MySQL, sqlc , TLS, sesiones, correo, respaldos, reloj e identificadores	Puertos definidos por capas internas	Decidir reglas funcionales del negocio
Cliente Wails	Interfaz, navegación, estado visual y consumo de API	Contratos HTTP y estado de pantalla	Seguridad exclusiva, cálculos finales o acceso a MySQL

4. Organización por módulos de negocio

No recomiendo una estructura global con carpetas como `controllers` , `services` , `models` y `repositories` que mezclen todos los módulos. A medida que crezcan las reglas, esa forma termina produciendo servicios enormes y dependencias cruzadas difíciles de controlar. Recomiendo **feature-first con capas internas**, de modo que cada módulo tenga cerca sus entidades, casos de uso, adaptadores y puertos.

Los módulos iniciales serían los siguientes:

Módulo	Responsabilidad principal	Agregados o conceptos relevantes
identity	Usuarios, roles, permisos, sesiones y autorización	User , Role , Permission , Session
catalog	Productos, categorías, unidades, códigos, precios y proveedores asociados	Product , Price , Category
sales	Venta, líneas, descuentos, estado y pagos aplicados	Sale , SaleLine , Payment
cash	Apertura, movimientos, cierre, diferencias y reapertura autorizada	CashSession , CashMovement , CashClose
inventory	Existencias, movimientos, conteos, ajustes, mermas y alertas	StockItem , StockMovement , InventoryCount
purchasing	Proveedores, órdenes, recepción parcial, lotes y cuentas por pagar	Purchase , Receipt , Payable

credit	Clientes, límites, plazos, saldos, abonos, vencimientos y bloqueo	CustomerAccount , CreditEntry , PaymentAllocation
returns	Anulaciones, devoluciones, cambios, reembolsos y saldos a favor	Return , Refund , CreditNote
billing	Comprobantes, impuestos, NCF/e-NCF parametrizable y notas de crédito	Document , TaxPolicy , FiscalNumber
reporting	Consultas de lectura, agregaciones, exportaciones y alertas	Read models y report queries
audit	Registro inmutable de acciones sensibles	AuditEntry
operations	Migración, respaldo, restauración y tareas operativas autorizadas	MigrationBatch , BackupJob

Estos módulos no deben interpretarse como servicios separados. En la primera versión se compilan y despliegan juntos dentro de la API Go, pero sus límites deben ser explícitos. Por ejemplo, `sales` puede solicitar una reserva o movimiento de inventario mediante un puerto; no debe escribir directamente en las tablas internas de `inventory`.

5. Estructura de carpetas recomendada

Una estructura adecuada para el backend sería la siguiente:

Plain Text

```
backend/
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── platform/
│   │   ├── config/
│   │   ├── httpserver/
│   │   ├── auth/
│   │   ├── mysql/
│   │   ├── transaction/
│   │   └── clock/
```

```

├── id/
├── mail/
├── backup/
├── logging/
├── modules/
│   ├── identity/
│   │   ├── domain/
│   │   ├── application/
│   │   │   ├── commands/
│   │   │   ├── queries/
│   │   │   └── ports/
│   │   ├── adapters/
│   │   │   └── http/
│   │   └── infrastructure/
│   │       └── mysql/
│   ├── catalog/
│   ├── sales/
│   ├── cash/
│   ├── inventory/
│   ├── purchasing/
│   ├── credit/
│   ├── returns/
│   ├── billing/
│   ├── reporting/
│   ├── audit/
│   └── operations/
├── shared/
│   ├── domain/
│   │   ├── money/
│   │   ├── pagination/
│   │   └── errors/
│   ├── application/
│   └── authorization/
├── db/
│   ├── migrations/
│   ├── queries/
│   └── generated/
│       └── sqlc/
├── contracts/
│   └── http/
├── Dockerfile
├── docker-compose.yml
├── go.mod
├── go.sum
├── client/
│   ├── frontend/
│   │   ├── src/

```

```

├── app/
│   ├── routes/
│   ├── features/
│   │   ├── pos/
│   │   ├── cash/
│   │   ├── inventory/
│   │   ├── purchases/
│   │   ├── credit/
│   │   └── reports/
│   ├── components/
│   ├── stores/
│   ├── services/
│   │   └── api/
│   ├── types/
│   └── lib/
├── package.json
├── pnpm-lock.yaml
├── app.go
├── api_client.go
└── wails.json

```

La carpeta `db/generated/sqlc` debe tratarse como código generado y no como dominio. Los tipos de `sqlc` se mapean a entidades o modelos de aplicación en los repositorios de cada módulo. Nunca se deben devolver directamente desde un handler HTTP ni utilizar directamente en Svelte.

6. Reglas de dependencia y contratos

El dominio debe ser el código más estable del sistema. Por ejemplo, una entidad `Sale` puede exponer operaciones como `AddLine`, `ApplyDiscount`, `RegisterPayment` y `Complete`, pero no debe recibir un `*sql.Tx`, un `http.Request` ni un tipo generado por `sqlc`.

La capa de aplicación debe definir puertos pequeños y orientados a casos de uso. Es preferible esto:

Go

```

type SaleRepository interface {
    FindDraft(ctx context.Context, id SaleID) (*Sale, error)
    Save(ctx context.Context, sale *Sale) error
}

type InventoryGateway interface {
    EnsureAvailable(ctx context.Context, items []InventoryRequest) error
    RegisterSaleMovement(ctx context.Context, movement StockSaleMovement) error
}

```

```
type CashGateway interface {
    RegisterSalePayments(ctx context.Context, sessionID CashSessionID, payments
}
```

que exponer una interfaz genérica como `Repository` con decenas de métodos para todas las tablas. Los puertos deben expresar una intención del negocio, no la forma de la base de datos.

6.1 Comandos y consultas

Recomiendo separar conceptualmente las operaciones que cambian estado de las operaciones que consultan información:

Tipo	Ejemplos	Característica
Comandos	CompleteSale , OpenCashSession , ReceivePurchase , RegisterCreditPayment , ApproveInventoryAdjustment , ProcessReturn	Validan reglas, ejecutan transacción y registran auditoría
Consultas	SearchProducts , GetCashSummary , GetCustomerBalance , ListLowStock , SalesReportByDate	No mutan estado y pueden usar consultas SQL optimizadas

Esto es un **CQRS ligero**, no dos sistemas separados. Los comandos deben usar entidades y puertos de dominio; las consultas de reportes pueden devolver read models específicos, contruidos con SQL diseñado para lectura. No conviene forzar entidades complejas para cada reporte.

7. Agregados y consistencia transaccional

La consistencia debe diseñarse alrededor de los invariantes reales del negocio, no alrededor de cada tabla. Una venta, por ejemplo, debe quedar en un estado válido antes de completarse: debe tener líneas, cantidades positivas, precios válidos, pagos compatibles con el total y una caja abierta cuando el medio de cobro lo requiera.

Agregado o componente	Invariantes principales
-----------------------	-------------------------

Sale	No se completa sin líneas; los totales se calculan en servidor; los descuentos respetan límites; una venta completada no se edita silenciosamente
CashSession	Solo una apertura activa por caja; el cierre compara esperado y contado; la reapertura exige autorización y auditoría
Inventory	Cada modificación genera un movimiento; la existencia negativa es explícita, advertida y reportable; una devolución solo repone mercancía vendible
CustomerAccount	El crédito respeta límite, plazo y bloqueo; cada cargo y abono genera un asiento; el saldo se puede reconstruir
PurchaseReceipt	Una recepción parcial solo registra lo recibido; la diferencia queda como incidencia; lote y vencimiento conservan trazabilidad
AuditLog	Las acciones sensibles son inmutables y contienen actor, fecha, operación, entidad, motivo y autorización

Una operación de negocio completa debe abrir una sola transacción MySQL en la capa de aplicación o mediante un `UnitOfWork / TransactionManager` . Por ejemplo, `CompleteSale` puede seguir este orden:

1. Validar identidad, permisos, caja activa, productos, precios, descuentos y crédito.
2. Crear la venta y sus líneas con importes calculados por el servidor.
3. Registrar pagos y movimientos de caja.
4. Registrar los movimientos de inventario.
5. Registrar el cargo de crédito si existe saldo pendiente.
6. Generar el comprobante o reservar la numeración correspondiente.
7. Registrar la auditoría.
8. Confirmar la transacción; si cualquier paso falla, revertir la operación completa.

El orden exacto debe ajustarse al modelo de datos, pero el principio es esencial: **la confirmación de venta no debe dejar una venta guardada sin inventario, un pago sin caja o un crédito sin saldo coherente.**

8. Decisiones técnicas importantes

8.1 Dinero, impuestos y fechas

No se deben utilizar `float32` ni `float64` para dinero. Recomiendo un objeto de valor `Money` basado en unidades menores con `int64`, acompañado de la moneda `DOP`. La base de datos puede almacenar importes como enteros en unidades menores o mediante `DECIMAL` con un mapeo explícito; lo importante es que el dominio no dependa de redondeos binarios.

Los impuestos deben encapsularse en una política o servicio de dominio parametrizable. La lógica fiscal no debe estar incrustada en componentes Svelte ni repartida en consultas SQL. Como la facturación NCF/e-NCF depende de validación contable y de un proveedor autorizado, conviene definir un puerto `FiscalDocumentProvider` y comenzar con un adaptador configurable que no obligue a integrar un proveedor externo antes de que esté formalmente aprobado.

Las fechas de negocio deben manejar la zona horaria de República Dominicana, mientras que los timestamps técnicos pueden almacenarse en UTC. La fecha de caja y la fecha del comprobante deben ser conceptos explícitos para evitar errores cerca de medianoche.

8.2 Auditoría y movimientos

La auditoría no debe ser un simple mensaje de log. Los logs técnicos sirven para depurar; la auditoría sirve para demostrar quién ejecutó una acción de negocio. Por ello, una `AuditEntry` debe incluir, como mínimo, actor, rol, fecha, tipo de acción, entidad afectada, identificador, motivo, autorización relacionada y un resumen de los valores anteriores y nuevos cuando corresponda.

Para ventas, inventario, caja y crédito recomiendo conservar libros de movimientos. En lugar de corregir silenciosamente el saldo, se registra un nuevo movimiento de ajuste, devolución, merma, abono o reapertura. Los saldos actuales pueden mantenerse como proyecciones para rendimiento, pero deben poder reconstruirse desde los movimientos.

8.3 Idempotencia y reintentos

La comunicación entre un cliente de escritorio y la API puede sufrir reintentos o pérdida de respuesta después de que el servidor haya confirmado una operación. Los comandos que crean efectos —venta, recepción, abono, devolución, retiro y cierre— deben aceptar una clave de idempotencia generada por el cliente. La API debe impedir que la misma operación se aplique dos veces.

Esta decisión es especialmente importante para un POS: si la empleada pulsa nuevamente “Completar venta” porque la respuesta tardó, no debe crearse una segunda venta ni

duplicarse el descuento de inventario.

9. Diseño del cliente Wails y Svelte

El cliente Wails debe ser deliberadamente delgado. Svelte controla pantallas, formularios, navegación, estados de carga y mensajes al usuario. Wails expone un cliente Go local que agrega autenticación de sesión, manejo de errores de transporte, timeout, reintentos seguros e invocación HTTPS. La autorización definitiva siempre permanece en la API.

La organización del frontend también debe ser por capacidades de negocio, no por una sola carpeta global de componentes:

Área frontend	Responsabilidad
features/pos	Carrito, búsqueda, lector, pagos mixtos, cambio y finalización
features/cash	Apertura, movimientos, cierre, diferencias y solicitudes de reapertura
features/inventory	Existencias, conteos, ajustes, mermas y alertas
features/purchases	Proveedores, compras, recepción parcial y cuentas por pagar
features/credit	Clientes, saldos, abonos, vencimientos y bloqueos
features/reports	Filtros, tablas, exportación y estados de carga
services/api	Cliente HTTP/Wails, serialización, sesión y errores de transporte
stores	Estado visual y de sesión, sin reglas financieras definitivas

El frontend puede mostrar totales preliminares para facilitar la interacción, pero el total autorizado debe ser recalculado por el backend. El cliente tampoco debe decidir si una empleada puede anular una venta, aprobar un ajuste o reabrir una caja; solo debe reflejar la respuesta de la API.

10. Autorización y seguridad

La seguridad debe aplicarse en tres niveles complementarios. Primero, autenticación de la sesión en la API. Segundo, autorización por permisos y rol para cada caso de uso. Tercero,

autorización contextual para operaciones sensibles, como descuentos fuera de límite, devolución con reembolso, ajuste de inventario, retiro extraordinario, reapertura de caja o restauración de respaldo.

La autorización no debe depender únicamente de ocultar botones. Un endpoint debe volver a validar permisos, límites, estado de caja, motivo y aprobador aunque el cliente no presente la acción en pantalla. La API Go es el único componente que debe tener credenciales de MySQL, de acuerdo con el baseline del proyecto. ¹

11. Reportes, alertas, correo y respaldos

Los reportes deben implementarse como consultas de lectura dentro del módulo `reporting`, con filtros por fecha y modelos específicos para cada informe. Las consultas de ventas, caja, inventario, compras, crédito y mermas no deben reconstruir toda la aplicación mediante llamadas entre módulos. Pueden leer vistas o consultas SQL optimizadas, siempre que no modifiquen el estado transaccional.

Las alertas de mínimos, vencimientos y morosidad deben separarse de la pantalla que las muestra. El dominio o la aplicación produce una condición de alerta; un adaptador de correo puede enviar el resumen a la administradora. Esta separación permite comenzar mostrando alertas dentro del POS y añadir el envío por correo sin cambiar las reglas.

Los respaldos y restauraciones son capacidades operativas, no reglas de venta. Deben vivir detrás de un puerto y ejecutarse con autorización administrativa, registro de auditoría, validación de integridad y un procedimiento de recuperación probado. La API puede coordinar la solicitud, pero el mecanismo concreto de backup debe quedar en infraestructura y operación del servidor.

12. Pruebas recomendadas

La cobertura debe priorizar las reglas que protegen dinero, inventario y trazabilidad. No basta con probar que un endpoint devuelve `200`.

Nivel	Qué probar	Herramientas compatibles
Dominio	Totales, descuentos, pagos mixtos, límites, vencimientos, estados, devoluciones vendibles y mermas	<code>go test</code>
Aplicación	Casos de uso con repositorios falsos, autorización, idempotencia y transacciones	<code>go test</code>

Adaptadores HTTP	JSON, códigos de error, autenticación, permisos y validación de entrada	<code>net/http/httptest</code> , <code>go test</code>
Integración	SQL real, migraciones, restricciones, bloqueos y rollback	MySQL con Docker Compose + <code>go test</code>
Frontend	Formularios, estados de carga, errores y componentes de pantalla	Vitest
Aceptación	Jornada completa: venta, compra, ajuste, merma, crédito, abono, devolución y cierre	Checklist de aceptación y ambiente de prueba

Los escenarios de aceptación definidos en el proyecto deben convertirse en pruebas automatizadas o guiones reproducibles, especialmente los relacionados con cierre de caja, recepción parcial, ventas a crédito, devoluciones y acciones sensibles. 2

13. Evolución futura

La arquitectura deja puntos de extensión sin convertir las funcionalidades futuras en dependencias prematuras. La impresión puede añadirse mediante un puerto

`ReceiptPrinter` ; la facturación electrónica avanzada mediante un `FiscalDocumentProvider` ; la integración bancaria mediante un adaptador de pagos; y la analítica mediante `read models` o un pipeline de datos separado cuando exista suficiente información confiable.

La futura multi-sucursal requerirá introducir explícitamente `branch_id` o una entidad equivalente en los agregados y políticas de acceso. No recomiendo simular esa capacidad completa desde la primera versión si el alcance aprobado es una sucursal; sí recomiendo evitar claves y nombres de tablas que hagan imposible añadirla después.

La extracción de un módulo a un servicio independiente solo debería considerarse cuando exista una necesidad real de escalado, despliegue independiente, aislamiento de riesgo o integración externa. Antes de extraerlo, el módulo debe tener contratos claros, ownership de sus datos y pruebas de integración que hagan visible el coste de la separación.

14. Antipatrones que conviene evitar

Antipatrón	Por qué sería perjudicial aquí	Alternativa
------------	--------------------------------	-------------

Reglas de negocio en Svelte	Se pueden evadir y no son compartidas por todos los clientes	Validación final en casos de uso de la API
Handlers HTTP que escriben directamente con <code>sqlc</code>	Mezcla transporte, persistencia y negocio	Handler delgado + caso de uso + repositorio
Un <code>service.go</code> gigante	Acumula ventas, caja, inventario y crédito sin límites	Servicios de aplicación por caso de uso y módulo
Modelo de base de datos como modelo de dominio	Cambiar tablas rompe reglas y contratos	Mapeadores entre persistencia y dominio
Eliminar o editar operaciones históricas	Destruye trazabilidad y dificulta cuadrar	Movimientos compensatorios, anulaciones y auditoría
Microservicios desde la primera versión	Añaden complejidad sin necesidad de escala	Monolito modular con límites internos
Reintentar comandos sin idempotencia	Puede duplicar ventas, pagos o recepciones	Claves de idempotencia y restricciones únicas
Usar <code>float64</code> para dinero	Produce errores de redondeo	<code>Money</code> con unidades menores y redondeo explícito
Confiar en ocultar botones	No constituye autorización real	Políticas verificadas en el backend

15. Orden de implementación sugerido

La implementación debería empezar por las piezas que establecen seguridad y consistencia, no por la pantalla más visible. Primero se deben fijar la configuración, identidad, autorización, errores, dinero, reloj, transacciones, auditoría y migraciones. Luego conviene implementar catálogo e inventario, porque ventas y compras dependen de ellos.

Fase	Entrega arquitectónica
1	Bootstrap de API, configuración, sesión, permisos, errores, migraciones, conexión MySQL, auditoría y pruebas base
2	Catálogo, inventario, movimientos y alertas de mínimos/vencimientos

3	POS, pagos, caja e idempotencia de comandos
4	Compras, recepción parcial, proveedores y cuentas por pagar
5	Crédito, abonos, morosidad, anulaciones, devoluciones y cambios
6	Facturación, reportes, exportaciones, correo, backups y migración histórica
7	Prueba de jornada completa, capacitación, aceptación y estabilización

Este orden mantiene el núcleo operativo pequeño y verificable, en línea con la decisión del proyecto de resolver primero el registro diario y el control operativo antes de incorporar impresión, integraciones complejas o analítica avanzada. 2

16. Conclusión

La arquitectura que recomendaría formalizar es, por tanto, la siguiente:

Un monolito modular Go, organizado por módulos de negocio y capas Clean Architecture, con casos de uso transaccionales, dominio independiente de frameworks, persistencia MySQL detrás de puertos, consultas de reporte separadas de comandos, clientes Wails delgados y auditoría/movimientos inmutables.

Esta solución ofrece el equilibrio correcto entre simplicidad presente y capacidad de evolución. No sobredimensiona un POS de una sucursal, pero tampoco convierte la primera versión en un conjunto de handlers y consultas inseparables. La prioridad debe ser que una venta, una recepción, un ajuste, un crédito, una devolución y un cierre de caja sean operaciones confiables, autorizadas, trazables y recuperables.

Referencias

[1] Stack tecnológico aprobado — Sistema POS e inventario para Provisiones Gurima

[2] Documento de alcance y requisitos — Sistema POS e inventario para Provisiones Gurima